Rotate the Image and Every Patch: A Self-Supervised Recipe for Vision Transformers

PatchRot trains a ViT to predict the rotation of both an image and each of its patches, and beats training from scratch across four datasets. Work from Arizona State University and Georgia State University.

Vision transformers (ViTs) have become a default backbone for image recognition, but they carry a well-known catch: they are data-hungry. A ViT treats an image as a sequence of patch tokens and lets self-attention decide how those tokens relate, which gives it enormous flexibility but almost none of the built-in assumptions a convolutional network gets for free. A ConvNet bakes in locality and translation structure; a ViT has to learn all of that from data. So when labels are scarce, ViTs tend to fall behind ConvNets, and they only pull ahead when trained on very large labeled datasets.

That makes self-supervised pretraining, learning useful features from unlabeled images before ever touching a labeled task, an obvious fit. The complication is that the popular self-supervised pretext tasks (jigsaw puzzles, colorization, image-rotation prediction) were all designed in the ConvNet era. A ConvNet emits a single global feature map, so those tasks ask a single global question about the whole image. They never exploit the one structural feature that makes a transformer a transformer: it produces a separate output for every patch.

PatchRot starts from that gap. The idea is simple enough to state in a sentence: rotate the whole image and rotate each of its patches by a random multiple of 90 degrees, and train the ViT to predict every rotation angle. The class token, which normally predicts the object category, predicts the whole-image rotation and so has to capture global structure; new per-patch heads predict each patch's own rotation and so have to capture local detail. Global and local supervision, from one label-free task, matched to the architecture.

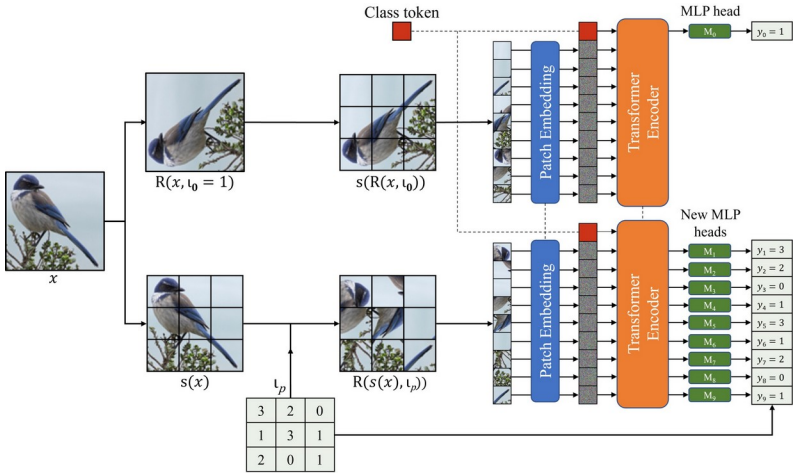

Figure 1. The PatchRot pretraining task. An input image runs through two shared-weight passes: in one the whole image is rotated by a random multiple of 90 degrees and the class token predicts that angle; in the other every patch is rotated independently and a new MLP head on each patch token predicts its angle. One label-

free task supervises global structure (via the class token) and local detail (via the patch heads) at the same time.

Paper: https://arxiv.org/abs/2210.15722

Code: https://github.com/s-chh/PatchRot

Why a transformer needs its own self-supervised task

The prior work PatchRot builds on is RotNet, which showed something counterintuitive: if you take a ConvNet and simply train it to predict how much an image has been rotated (0, 90, 180, or 270 degrees), it learns surprisingly rich features, because guessing the rotation forces the network to recognize what objects look like when upright. RotNet asks one question per image. A ViT, however, can answer a question per patch. The natural move is to push rotation prediction down to the patch level, and that patch-level signal is exactly what a token-based model is built to consume, yet no earlier self-supervised method had tried it.

How PatchRot works

At pretraining time PatchRot runs two tasks. In the image-rotation task the whole image is rotated by a random multiple of 90 degrees, and the class token predicts that single angle as a 4-way classification. In the patch-rotation task each patch is rotated independently, and a dedicated MLP head attached to each patch token predicts that patch's angle, again 4-way. The two tasks are applied in separate forward passes rather than at once, and the passes share the same encoder weights, so the transformer learns both signals without having to disentangle them from a single scrambled input.

The division of labor is the point. The class token aggregates information across the whole image, so asking it for the image's rotation nudges it toward global structure. The patch heads see individual tokens, so asking each one for its patch's rotation nudges the model toward local detail. After pretraining, the extra patch heads are simply discarded; only the class-token head is kept and fine-tuned on the real classification task.

There is one subtlety that makes patch rotation actually hard. If patches are cut directly next to each other, the model can cheat: it can line up continuous edges between neighboring patches and read off the rotation without understanding any content. PatchRot blocks this by carving patches from a slightly larger grid and randomly cropping them, leaving a random buffer gap (the paper sets the buffer to a quarter of the patch size) between neighbors. With the seams broken, edge continuity no longer gives the answer away, and the model has to learn genuine local structure.

One more practical detail: self-supervised training runs at a reduced resolution to keep it cheap, and downstream fine-tuning switches to the original resolution, interpolating the positional embeddings to the larger number of patches. The backbone throughout is a compact ViT with 6 transformer encoder blocks, evaluated at 32×32 for CIFAR and FashionMNIST and 64×64 for Tiny-ImageNet.

Does it work? Fine-tuning from any depth

The cleanest way to judge a pretrained feature extractor is to ask how little of it you need to fine-tune. Figure 2 sweeps exactly that: the x-axis walks from freezing almost everything (NF, and MLP, which trains only the final head, i.e. linear probing) to fine-tuning progressively deeper blocks. Each panel is one dataset, and three curves compare PatchRot, RotNet, and a supervised-from-scratch baseline.

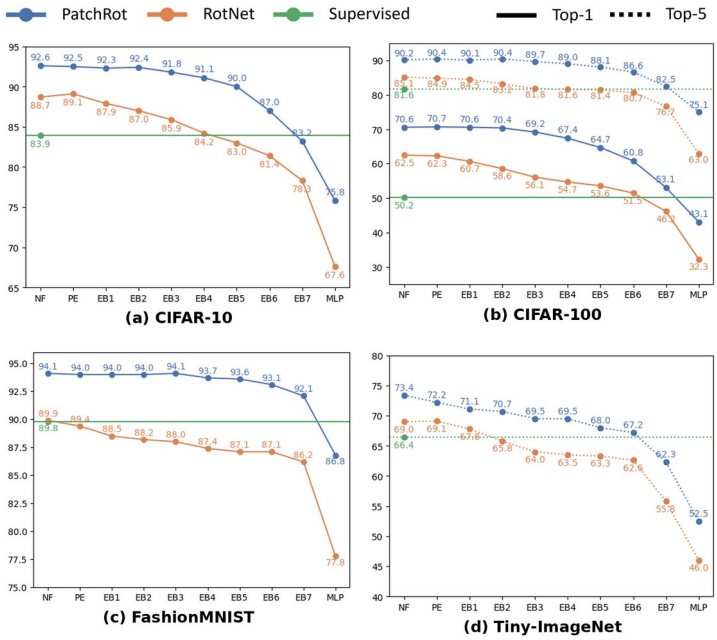

Figure 2. Accuracy as a function of how much of the ViT is fine-tuned (NF and MLP freeze the most; the encoder blocks EB1-EB7 progressively unfreeze) on CIFAR-10, CIFAR-100, FashionMNIST, and Tiny-ImageNet. The PatchRot curve sits above both RotNet and the supervised baseline at essentially every depth. Notably, even linear probing (MLP) approaches supervised accuracy, and fine-tuning just a single encoder block already surpasses training from scratch.

Two things stand out. First, the PatchRot curve sits above RotNet everywhere, and above the supervised-from-scratch line across the useful range, on all four datasets. Second, and more telling, the gains survive heavy freezing: linear probing alone (training only the final layer on top of frozen PatchRot features) gets close to the fully supervised number, and fine-tuning a single encoder block already beats a network trained from scratch. That is the signature of features that are genuinely useful rather than a fine-tuning artifact.

The headline numbers, taken at full fine-tuning, make the size of the effect concrete.

Table 1. PatchRot pretraining versus supervised training from scratch, at full fine-tuning.

Dataset	Metric	Supervised	PatchRot	Gain
CIFAR-10	Top-1	83.9	92.6	+8.7
CIFAR-100	Top-1	50.2	70.6	+20.4
CIFAR-100	Top-5	81.6	90.2	+8.6
FashionMNIST	Top-1	89.8	94.1	+4.3
Tiny-ImageNet	Top-5	66.4	73.4	+7.0

The jump is largest where the task is hardest and labels are spread thinnest: on CIFAR-100, PatchRot lifts top-1 accuracy from 50.2 to 70.6, a gain of over 20 points, while on the

easier CIFAR-10 it still adds 8.7 points. In every case the same compact ViT is doing the work; the only difference is whether it started from scratch or from PatchRot features.

What the transformer actually learns

Numbers say the features help; attention maps hint at why. Because PatchRot never sees a class label during pretraining, there is no obvious reason for it to care about objects, yet it learns to.

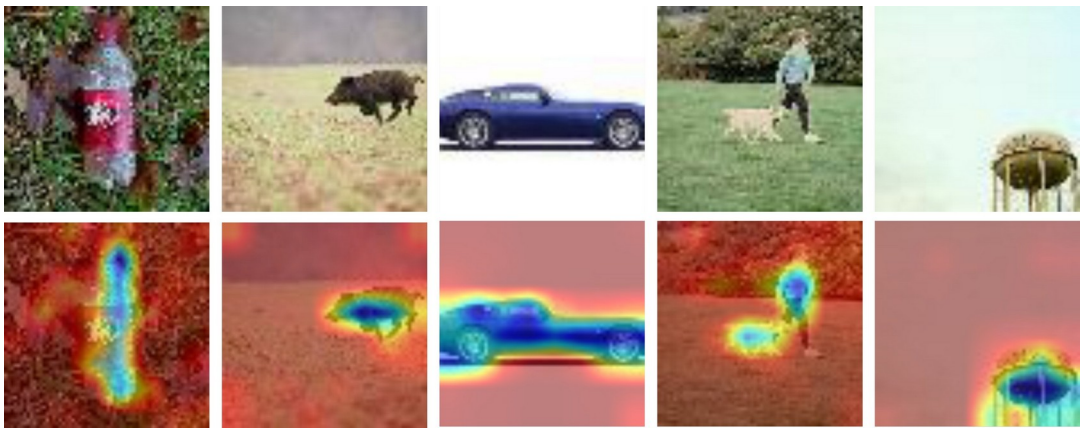

Figure 3. Attention maps from a ViT pretrained with PatchRot on Tiny-ImageNet (top row: input images; bottom row: the model's attention). Without ever seeing a label, the model concentrates its attention on the salient object in each scene, evidence that rotation prediction pushes it toward object-centric features.

The maps concentrate on the salient object in each image, the bottle, the boar, the car, the person and dog, the water tower, rather than scattering across the background. This is a plausible reason the features transfer so well: to tell which way a patch has been turned, the model has to know what the thing in that patch is, and that knowledge is precisely what a downstream classifier wants.

More pretraining, better features

If the self-supervised signal is real, more of it should keep paying off. Figure 4 varies the number of PatchRot pretraining epochs on CIFAR-100 and tracks downstream accuracy at several fine-tuning depths.

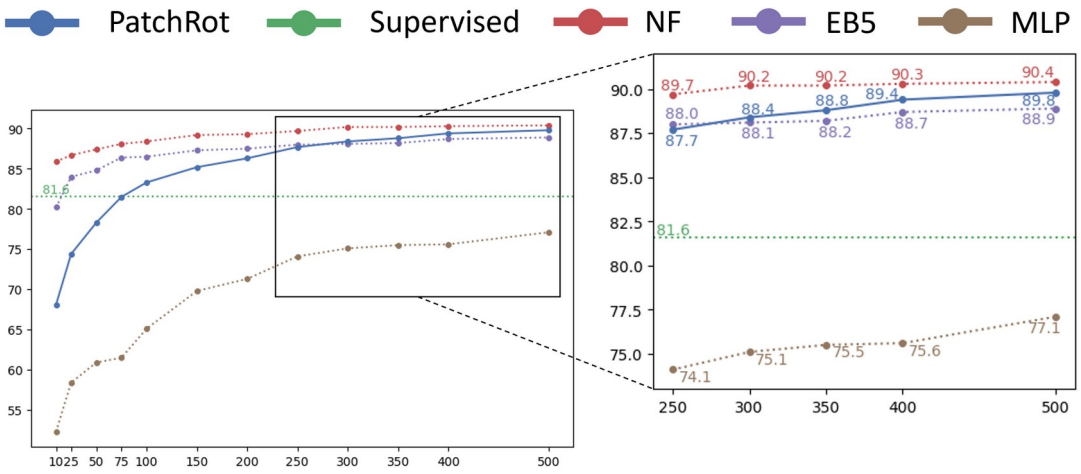

Figure 4. Downstream accuracy on CIFAR-100 as the amount of PatchRot pretraining grows (the right panel zooms into the later epochs). Across fine-tuning depths (NF, EB5, MLP) more self-supervised pretraining steadily improves accuracy, and the frozen-feature curves stay well above the supervised baseline of 81.6.

Accuracy climbs steadily with pretraining and then flattens, exactly the diminishing-returns curve you want to see: the gains are not a lucky early-stopping artifact but accumulate as the model spends more time on the pretext task. Even with most of the network frozen, the PatchRot curves sit comfortably above the supervised baseline, reinforcing that the value is in the learned features rather than in the fine-tuning.

Features that transfer, and that rescue low-label settings

A good pretraining task should also produce features that move to a new dataset. The authors test this by pretraining with PatchRot on one dataset and fine-tuning on another, in both directions between CIFAR-10 and CIFAR-100.

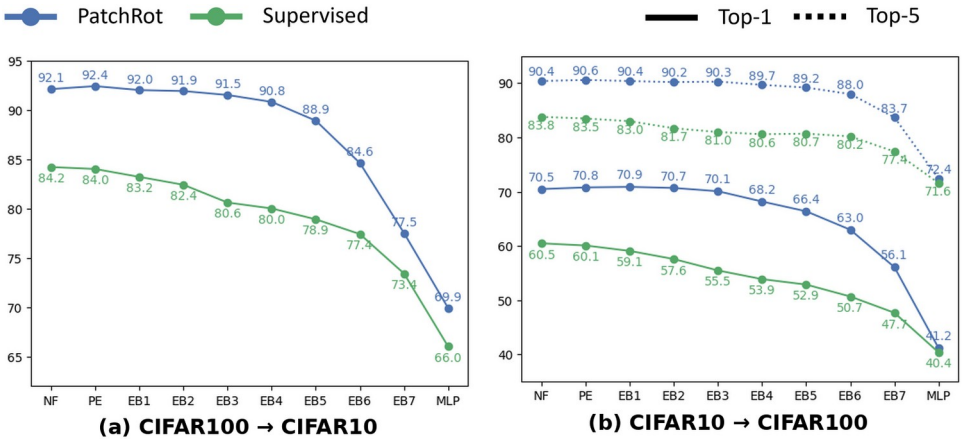

Figure 5. Transfer learning between CIFAR-100 and CIFAR-10 in both directions. Across every fine-tuning depth, features pretrained with PatchRot on the source dataset (blue) transfer better than supervised pretraining on the same source (green), showing the learned representation is not narrowly tied to one label set.

In both transfer directions the PatchRot curve stays above supervised pretraining at every depth, so the representation is not narrowly tuned to one label set. The same strength shows up

when labels are scarce: in a semi-supervised setting on CIFAR-10 with only 4000 labeled images, PatchRot reaches 81.1 accuracy against 53.7 for supervised training on the same handful of labels. When labels are the bottleneck, label-free features are worth the most.

Which pieces matter?

PatchRot has a few moving parts, so the authors ablate them on CIFAR-10. Removing either half of the supervision hurts: predicting only patch rotations (dropping the whole-image signal) or only the image rotation (essentially RotNet ported to a ViT) both land below the full method. The other design choices, keeping image and patch rotations in separate passes, pretraining at reduced resolution, and adding fresh patch heads instead of reusing the class-token head, each contribute a smaller but consistent gain: rotating the image and its patches together instead of in separate passes drops accuracy to 90.7, and pretraining at the original resolution rather than a reduced one gives 92.1, both short of the full method.

Table 2. Ablation on CIFAR-10 (top-1 accuracy). Every component of PatchRot contributes.

Variant	Top-1
Full PatchRot	92.6
No image rotation (patch only)	91.8
No patch rotation (image only)	91.0
Rotate image and patches together	90.7
Pretrain at original resolution	92.1
Reuse class-token head for patches	91.1

Takeaway

PatchRot is a reminder that a self-supervised task works best when it is shaped to the architecture it trains. Predicting the rotation of a whole image is an old idea; the new part is predicting the rotation of every patch as well, which turns a single global question into the dense, per-token signal a transformer is built to answer. The class token learns global structure, the patch heads learn local detail, and the resulting features beat both supervised-from-scratch and RotNet across four datasets, transfer cleanly between datasets, and pay off most when labels are scarce, all with cheap, general components (a buffer gap between patches and reduced-resolution pretraining) rather than heavy machinery.

The honest boundary is in the premise: PatchRot leans on rotation being informative, so its edge is thinnest on categories where orientation carries little meaning, which is why the authors separately probe rotation-invariant objects like the digits in SVHN and MNIST. For the common case of natural images with a clear upright, though, it is a lightweight, practical recipe for pretraining vision transformers when labeled data is in short supply.